

Geometric Foundations of Data Analysis I

Joshua Maglione

November 24, 2025

Contents

4	Clustering	1
4.1	Introduction to k -means clustering	3
4.2	Image compression with k -means	3
4.3	The k -means algorithm	3
4.4	A small example with k -means	4
4.5	Introduction to hierarchical clustering	5
4.6	An application of hierarchical clustering	5
4.7	The neighbour joining algorithm	6
4.8	A small example of hierarchical clustering	8

4 Clustering

Similar to PCA, clustering is a way to understand structure of data. Clustering or cluster analysis is a generic term for algorithms designed to partition a given data into clusters. Clusters is a word that does not really have a precise meaning, and its meaning might even depend on situation, but generally we intuitively understand that clusters are groups of data points as seen in Figure 4.1.

Clustering is a fantastic tool for answering certain kinds of questions about the data like the following.

- What sub-populations exist in the data?
- Are there outliers?
- Are the sub-populations cohesive or can they be partitioned further?

“Populations” presupposes a certain perspective that need not be true. For example, clustering algorithms are used on gene sequences in medicine and biological sciences. See for example the hierarchical clustering done on sequences of genes for breast cancer in Figure 4.2.

Clustering is also one of the first steps in machine learning—an example of unsupervised learning. This just means that the machine “learns” of patterns and structures from the unlabeled data. Sometimes data are put into clusters,

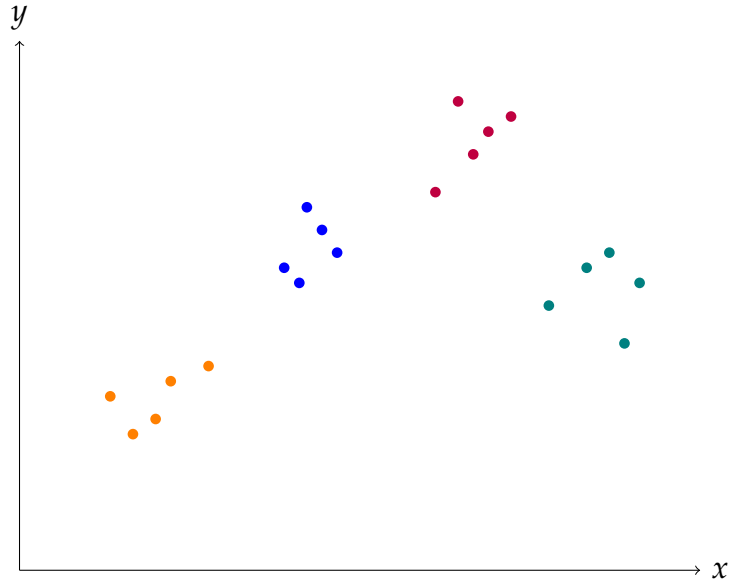


Figure 4.1: Data points in $\mathbb{R}^2$ that form four clusters.

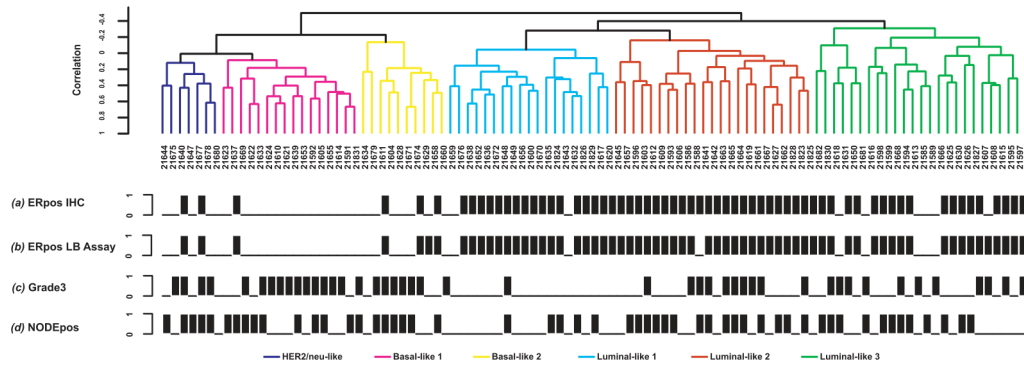


Figure 4.2: A hierarchical figure taken from [1]. The clusters are seen with the different colours.

and then nearly all of the data, except for the cluster ID is discarded, which can save significant space in the analysis stage. To quote Google [3]:

At Google, clustering is used for generalization, data compression, and privacy preservation in products such as YouTube videos, Play apps, and Music tracks.

There is no one clustering algorithm that is optimal in every situation; in fact, because data can cluster in many different ways, one needs a whole clustering toolkit. We will talk about two common methods for clustering: *k-means* and *single linkage* clustering. At the core of both of these clustering algorithms is a different assumption on the definition of a cluster.

4.1 Introduction to k -means clustering

The idea around k -means clustering is incredibly simple: group the data into k clusters based on the shortest distance to the center of the cluster. From this, we see how “cluster” is interpreted in the k -means algorithm—a cluster forms a “cell” in Euclidean space and data points outside of this cell are outside of the cluster. The shape of these cells depend on the layout of the centers of the clusters.

Before we go through the algorithm for k -means clustering, let’s explore some of the many applications of this algorithm.

4.2 Image compression with k -means

Suppose we have a jpg file that is $m \times n$ pixels. We can turn this image file into a 3-dimensional array, namely an $m \times n \times 3$ array. Alternatively, we can interpret this as three $m \times n$ matrices corresponding to the red, green, and blue values of the image.

We can use k -means clustering by interpreting each pixel as a data point in $\mathbb{R}^3$ and determining clusters. Once we find our clusters, we can replace all the pixels with the center it is closest to. The impact is that we replace all the different colours with exactly k colours—essentially compressing the image.

Let’s see how this might look on one of my photos. Meet my dog, Sherlock; he is depicted in Figure 4.3. There is not very many colours in the image, so we should expect that low values of k will produce an image that looks fairly similar. Running a k -means clustering for each $k \in \{2, \dots, 10\}$ yields nine different images seen in Figure 4.4.

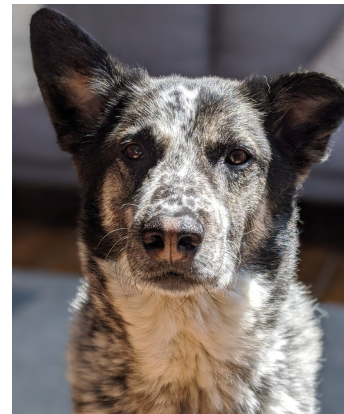


Figure 4.3: Sherlock.

4.3 The k -means algorithm

There are many versions of the basic algorithm we will describe. We use pseudo-code to give the details of the algorithm; this is a way of giving the idea of the structure of the code without diving into the specifics of any particular language.

We provide pseudo-code for a basic k -means algorithm in Algorithm 2. The algorithm returns a function $\Lambda : \{1, \dots, n\} \rightarrow \{1, \dots, k\}$, which can be interpreted as a label for the n data points. The algorithm accomplishes this by randomly choosing the k centroids, and then clustering the data points accordingly. Once all the data points have been labeled, we recompute the centroids by taking the mean point of all the data points in a given cluster to be the new centroid.

In the first step of k -means (see line 1 in Algorithm 2), we randomly assign the k centroids. For different initial assignments, one can get different outputs. This is like trying to find global extrema on a “bumpy” graph by starting at a random point on the graph and then looking for the nearest local extrema. There are a



Figure 4.4: k -means clustering on the image in Figure 4.3.

number of ways to improve the k -means algorithm, we will not cover them, but many consider different variations for their specific data at hand.

4.4 A small example with k -means

We will consider 40 points in $\mathbb{R}^2$, which can be seen in Figure 4.5, and we will look for 4 clusters. You might try to see how you would cluster the data.

We will randomly select four data points for our initial centroids, and then we will group the data points into the four clusters. We will show this by colouring the Voronoi cells different colours and indicating the centroids by white crosses. All of the iterations of one k -means algorithm can be seen in Figure 4.6. Just to illustrate how sensitive k -means is to the initial choice of centroids, we run the

Algorithm 2 (Basic) k -means

Input: a positive integer k and n data points $x_i \in \mathbb{R}^m$,

Output: a map $\Lambda : \{1, \dots, n\} \rightarrow \{1, \dots, k\}$.

```
1: Randomly assign the  $k$  centroids  $c_i$  to  $k$  data points.
2: Let  $\Lambda_0$  and  $\Lambda$  be constant functions mapping to  $-1$  and  $0$ , respectively.
3: while  $\Lambda_0 \neq \Lambda$  do
4:   Set  $\Lambda_0 = \Lambda$ .
5:   for  $i \in \{1, \dots, n\}$  do
6:     Set  $\Lambda(i) = j$  if  $\|x_i - c_j\| = \min\{\|x_i - c_r\| : 1 \leq r \leq k\}$ .
7:   end for
8:   for  $i \in \{1, \dots, k\}$  do
9:     Set  $c_i = \left( \sum_{j \in \Lambda^{-1}(i)} x_j \right) / \#(\Lambda^{-1}(i))$ .
10:  end for
11: end while
12: return  $\Lambda$ .
```

algorithm again in Figure 4.7.

4.5 Introduction to hierarchical clustering

Hierarchical clustering is, in some sense, the opposite approach to clustering than k -means. We will primarily be working with Single-linkage clustering. Single-linkage or, more generally, *neighbour joining* clustering is a hierarchical clustering algorithm. With k -means, we need to give the number of clusters k in order to run the algorithm. With neighbour joining, we look at all the possible clusterings under prescribed conditions by means of a *dendrogram*. We have already seen an example of the output of a hierarchical clustering algorithm; see Figure 4.2.

The basic idea of neighbour joining clustering is that every data point is its own cluster and under certain conditions, two clusters are merged at each step until all points are in one cluster. The output is not a specific clustering of the data, but rather all the possible clusterings. This is communicated through a dendrogram.

Before we look at the algorithm, let's consider a specific application.

4.6 An application of hierarchical clustering

We will look at the study in [1] a little closely; see also Figure 4.2. People diagnosed with breast cancer can have very different clinical outcomes. We do not really know why this is the case, but one suggestion might be that our knowledge of the specific breast cancer tumors is lacking. Sotiriou et al. [1] propose clustering the tumors by gene expression profiles, via unsupervised hierarchical clustering.

Prior to their work, the tumors could be partitioned into two major subgroups based on their estrogen receptors. Using clustering methods, Sotiriou et al. were able to obtain this partition at the top of the dendrogram—if one cuts it so there

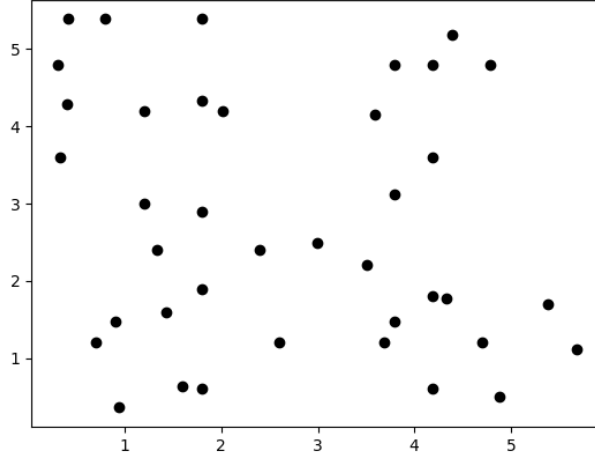


Figure 4.5: The plotted data we will use to run a 4-means clustering.

are exactly two clusters, these align with the two subgroups based on estrogen receptors. They were able to further partition the two subgroups based on their basal and luminal characteristics. This led to the six clusters (represented by different colours) in Figure 4.2. This is also in line with previous work done by different research teams, and offers more evidence that their hypothesis is correct—that breast cancer tumors ought to be clustered in the established way.

4.7 The neighbour joining algorithm

We will describe a class of *agglomerative* algorithms that can be altered to produce different outcomes based on certain given conditions. In order to do this, one of the inputs is a distance function, which can be obtained from a *metric*. We want the metric to be defined on finite subsets of $\mathbb{R}^m$, which will be interpreted as clusters in the actual algorithm.

Definition 4.1. A function $d : \mathcal{M} \times \mathcal{M} \rightarrow \mathbb{R}$ on a set $\mathcal{M}$ is a *metric* if

1. for all $X, Y \in \mathcal{M}$, we have $d(X, Y) \geq 0$, where equality holds if and only if $X = Y$,
2. for all $X, Y \in \mathcal{M}$, we have $d(X, Y) = d(Y, X)$, and
3. for all $X, Y, Z \in \mathcal{M}$, the triangle inequality is satisfied:

$$d(X, Y) + d(Y, Z) \geq d(X, Z).$$

Functions satisfying (1) and (2) are called *semimetrics*.

Lemma 4.2. The Euclidean distance $d(x, y) = \|x - y\|$ is a metric on $\mathbb{R}^m$. The square of the Euclidean distance, $d_{\square}(x, y) = \|x - y\|^2$, is not a metric but a semimetric.

Proof. To show that $d_{\square}$ does not satisfy the triangle inequality, let $a \in \mathbb{R}$ be positive. Then $|-a|^2 + |a|^2 < |2a|^2$, so

$$d_{\square}(a, 0) + d_{\square}(-a, 0) < d_{\square}(a, -a). \quad \square$$

In the context of clustering, having a metric is not necessary. For example, because $x \mapsto x^2$ is monotonically increasing on the $\mathbb{R}_{\geq 0}$, it follows that $d(x, y) < d(u, v)$ if and only if $d_{\square}(x, y) < d_{\square}(u, v)$; here we are using the notation from Theorem 4.2. Thus, it is irrelevant if we cluster two points based on d or $d_{\square}$, so we allow our distance functions to come from semimetrics.

Let's write $\mathcal{F}_m$ for the set of finite subsets of $\mathbb{R}^m$, so we will describe a few distance functions on $\mathcal{F}_m$.

Single-linkage: For $C_1, C_2 \in \mathcal{F}_m$,

$$d(C_1, C_2) = \min \{ \|x - y\| : x \in C_1, y \in C_2 \}.$$

Complete-linkage: For $C_1, C_2 \in \mathcal{F}_m$,

$$d(C_1, C_2) = \max \{ \|x - y\| : x \in C_1, y \in C_2 \}.$$

Average-linkage: For $C_1, C_2 \in \mathcal{F}_m$,

$$d(C_1, C_2) = \frac{1}{\#C_1 \cdot \#C_2} \sum_{x \in C_1} \sum_{y \in C_2} \|x - y\|.$$

Technically the single-linkage is not a semimetric: if $C_1 \cap C_2 \neq \emptyset$ yet $C_1 \neq C_2$, then $d(C_1, C_2) = 0$. However we only consider sets C_1 and C_2 with trivial intersection anyways, so this problem is avoided.

Lemma 4.3. *The complete-linkage function defines a metric on $\mathcal{F}_m$.*

There are of course other distances one could use. Single-linkage clustering, or nearest neighbour, corresponds to neighbour joining with the single-linkage metric. Likewise, complete-linkage clustering, or farthest neighbour, corresponds to neighbour joining with the complete-linkage metric. One could design a metric that might be most relevant for the particular data set at hand.

The output for a hierarchical clustering algorithm is a *dendrogram* or data equivalent to one. Figure 4.2 contains one example. Another very similar example is a phylogenetic tree seen in Figure 4.8.

To read a dendrogram, one first finds the many leaves or ends—often these are labeled—these are the initial clusters. As one goes through the dendrogram (e.g. bottom to top or left to right), clusters get linked, which is depicted by joining the ends of the initial clusters. This happens until there is exactly one cluster left. Two ends are closer (or more similar) if they become joined early on compared to two ends that only become joined near the end of the dendrogram.

Let's explain the algorithm using pseudo-code. Instead of outputting a dendrogram, we will output a list of pairs of the form $(t, [C_1, \dots, C_r])$, where $t \geq 0$

Algorithm 3 Neighbour joining

Input: a semimetric $d : \mathcal{F}_m \times \mathcal{F}_m \rightarrow \mathbb{R}$ and n data points $x_i \in \mathbb{R}^m$,

Output: a list D equivalent to a dendrogram.

```
1: Set  $t = 0$ .
2: Set  $C = \{\{x_1\}, \dots, \{x_n\}\}$ .
3: Set  $D = [(t, C)]$ .
4: while  $\#C > 1$  do
5:   Construct distance matrix  $M = (M_{ij})$ , with  $M_{ij} = d(C_i, C_j)$ .
6:   Determine the two closest distinct clusters  $C_r$  and  $C_s$  from  $M$ .
7:   Set  $t = t + M_{rs}$ .
8:   Set  $C = (C \setminus \{C_r, C_s\}) \cup \{C_r \cup C_s\}$ .
9:   Append to  $D$  the pair  $(t, C)$ .
10: end while
11: return  $D$ .
```

and each C_i is a finite set of data points. This list will start with $(0, [C_1, \dots, C_n])$, where each C_i is a singleton, and it will end with $(t_\infty, [C_1])$, where C_1 contains all of the data points. Note that this data is also equivalent to a dendrogram.

We will soon go through Algorithm 3 on a small example. Unlike k -means, neighbour joining is *deterministic*; that is, it runs the same every time on the same input. Notice in Algorithm 3 that the actual points x_i do not matter; instead, it's the distances between the points (and clusters) that matter. Thus, we won't specify points but a *graph*.

Definition 4.4. A (finite, simple) **graph** G is a pair of vertices $V = \{v_1, \dots, v_n\}$ and edges $E = \{e_1, \dots, e_k\}$, where each $e_i \subseteq V$ of size exactly two. Vertices v_i and v_j are **adjacent**, written $v_i \sim v_j$, if $\{v_i, v_j\} \in E$.

Graphs are ubiquitous in mathematics and computer science, and they can be easily visualised (for a relatively small amount of vertices). Three graphs can be seen in Figure 4.9.

4.8 A small example of hierarchical clustering

Instead of writing down or plotting data points, we will write down a distance matrix. The distance matrix is a symmetric matrix with zeros along the diagonal. (Why?) The (i, j) entry is equal to the distance between vertices (or clusters) v_i and v_j . Let's consider the following distance matrix.

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	4	8	1	7	8
v_2	4	0	6	5	5	6
v_3	8	6	0	7	2	2
v_4	1	5	7	0	8	8
v_5	7	5	2	8	0	1
v_6	8	6	2	8	1	0

We will apply single- and complete-linkages and cluster the data accordingly. Because we know how to compute them both given the distances from all pairs of vertices, we will not recompute our distance matrix each time. Instead we will draw nine graphs labeled G_t for $t \in \{0, \dots, 8\}$ since the maximal distance between any two points is 8. We will include an edge between v_i and v_j in G_t if and only if $d(v_i, v_j) \leq t$. The graph G_0 is the graph on 8 vertices with no edges, and the graph G_8 is the graph on 8 vertices where every vertex is adjacent to every other vertex (this is called a *complete graph*). All of the graphs are shown in Figure 4.10.

Now we will produce dendrograms, one for the single-linkage and one for the complete-linkage. There are two concepts in Graph Theory that we can use to determine these dendrograms.

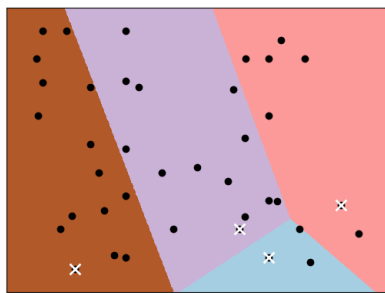
Definition 4.5. Two vertices v_i and v_j of a graph G are **connected** if there exists a sequence of adjacent vertices of the form $v_i = v_0 v_1 \dots v_\ell = v_j$.

Definition 4.6. A subset S of vertices of a graph G form a **clique** if for all distinct $v_i, v_j \in S$, the edge $\{v_i, v_j\}$ is in G .

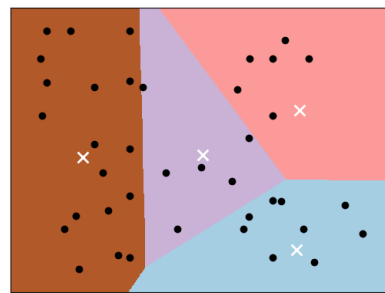
Let's consider the graphs in Figure 4.10 again. Using single-linkage, we know that vertices v_i and v_j lie in the same cluster if and only if v_i and v_j are connected. Therefore, we have exactly one cluster starting at G_5 using single-linkage. Using complete-linkage, we know that vertices $\{v_{i_1}, \dots, v_{i_\ell}\}$ form a cluster if and only if they form a clique. Hence, we have exactly one cluster starting at G_8 using complete-linkage. The dendrograms can be seen in Figures 4.11 and 4.12.

References

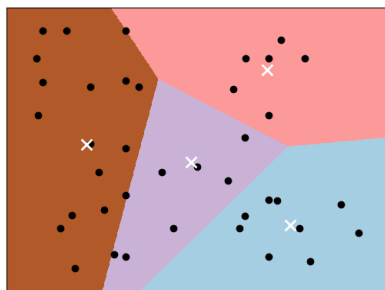
- [1] Christos Sotiriou, Soek-Ying Neo, Lisa M. McShane, Edward L. Korn, Philip M. Long, Amir Jazaeri, Philippe Martiat, Steve B. Fox, Adrian L. Harris, and Edison T. Liu, *Breast cancer classification and prognosis based on gene expression profiles from a population-based study*, Proceedings of the National Academy of Sciences **100** (2003), no. 18, 10393–10398.
- [2] Peter Frankl and Hiroshi Maehara, *Some geometric applications of the beta distribution*, Ann. Inst. Statist. Math. **42** (1990), no. 3, 463–474.
- [3] Google, *What is Clustering?* (2023), date accessed: 16 Oct. 2023. <https://developers.google.com/machine-learning/clustering/overview>.
- [4] Sarel Har-Peled, Piotr Indyk, and Rajeev Motwani, *Approximate nearest neighbor: towards removing the curse of dimensionality*, Theory Comput. **8** (2012), 321–350, DOI 10.4086/toc.2012.v008a014. MR2948494
- [5] Dan Margalit and Joseph Rabinoff, *Interactive Linear Algebra*, 2019, <https://textbooks.math.gatech.edu/ila/>.
- [6] Jonathon Shlens, *A tutorial on principal component analysis*, 2014, [arXiv:1404.1100](https://arxiv.org/abs/1404.1100).
- [7] Fan Yang and Sifan Liu and Edgar Dobriban and David P. Woodruff, *How to reduce dimension with PCA and random projections?*, 2021, [arXiv:2005.00511](https://arxiv.org/abs/2005.00511).



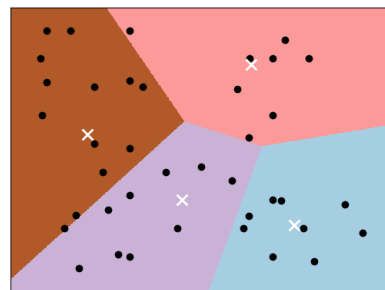
(a) Iteration 0



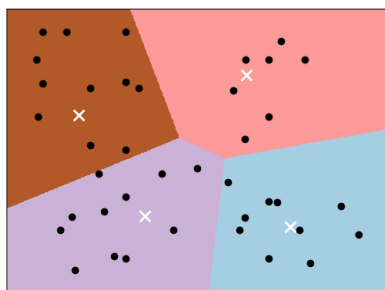
(b) Iteration 1



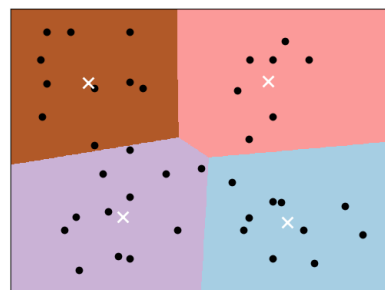
(c) Iteration 2



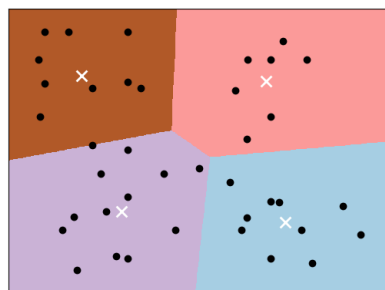
(d) Iteration 3



(e) Iteration 4



(f) Iteration 5



(g) Iteration 6

Figure 4.6: Seven iterations of one k -means algorithm.

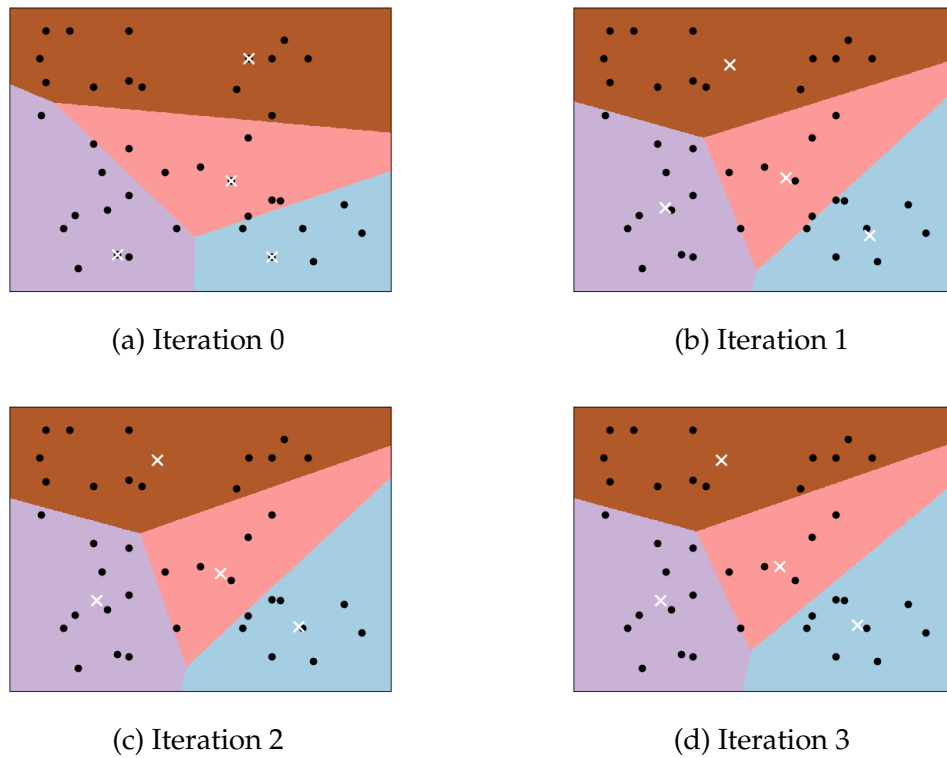


Figure 4.7: Four iterations of another k -means algorithm.

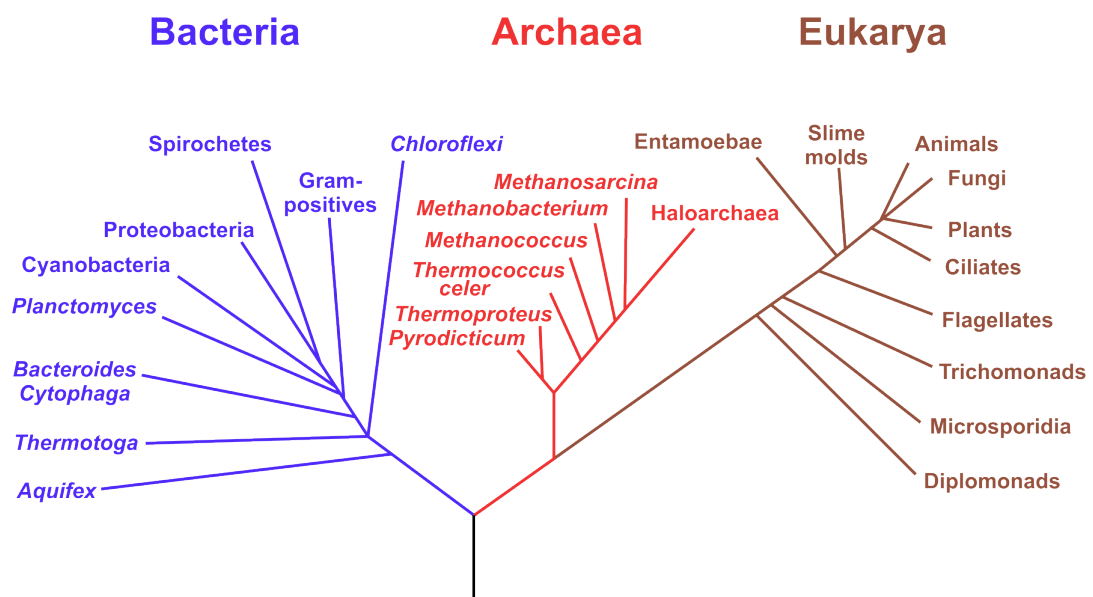


Figure 4.8: A phylogenetic tree is equivalent to a dendrogram.

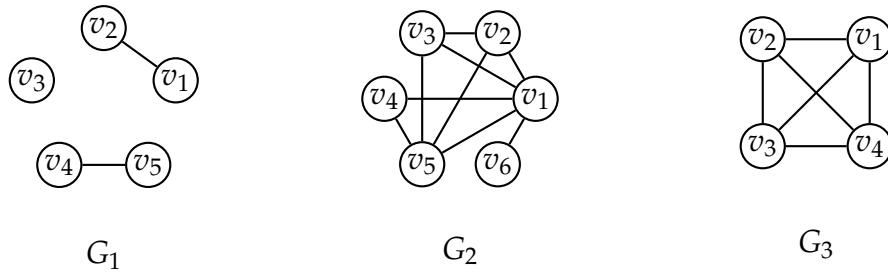


Figure 4.9: Three different graphs.

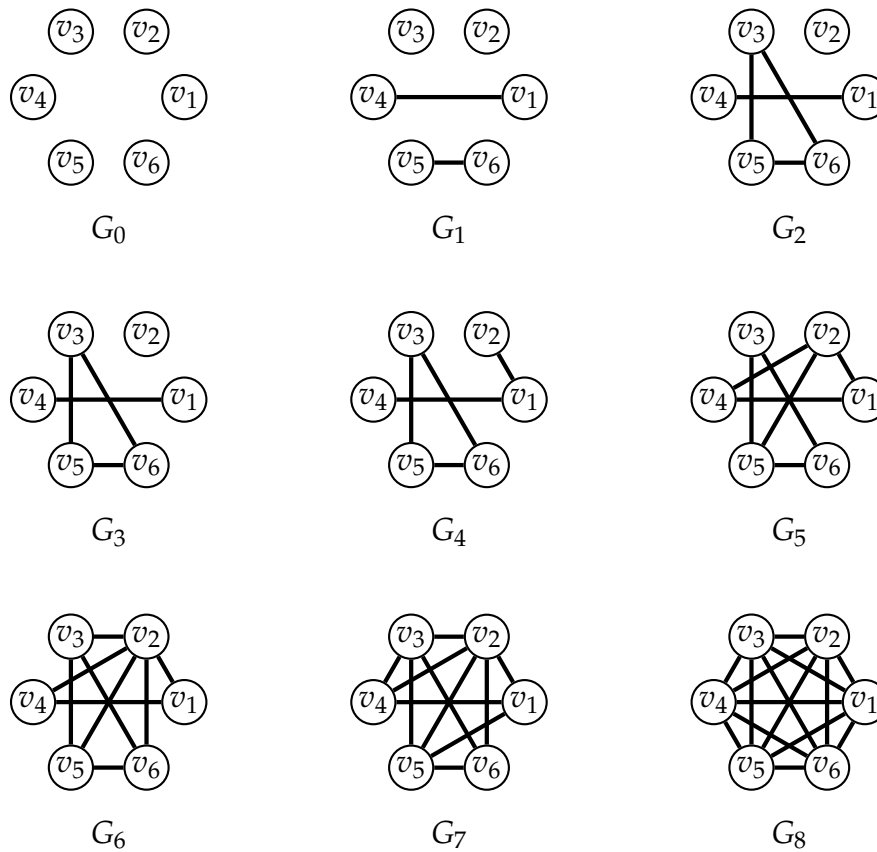


Figure 4.10: The 9 graphs visualising the possible clustering of the 8 vertices.

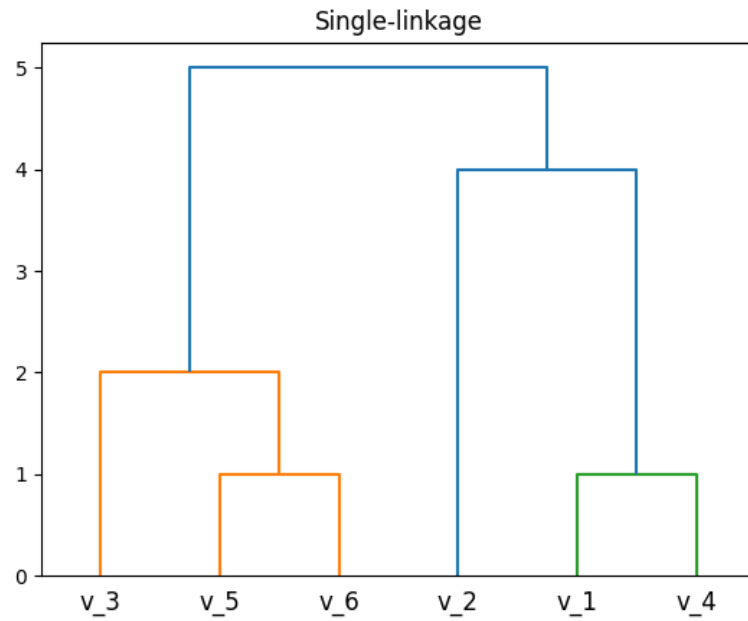


Figure 4.11: The dendrogram corresponding to the graphs in Figure 4.10 using single-linkage.

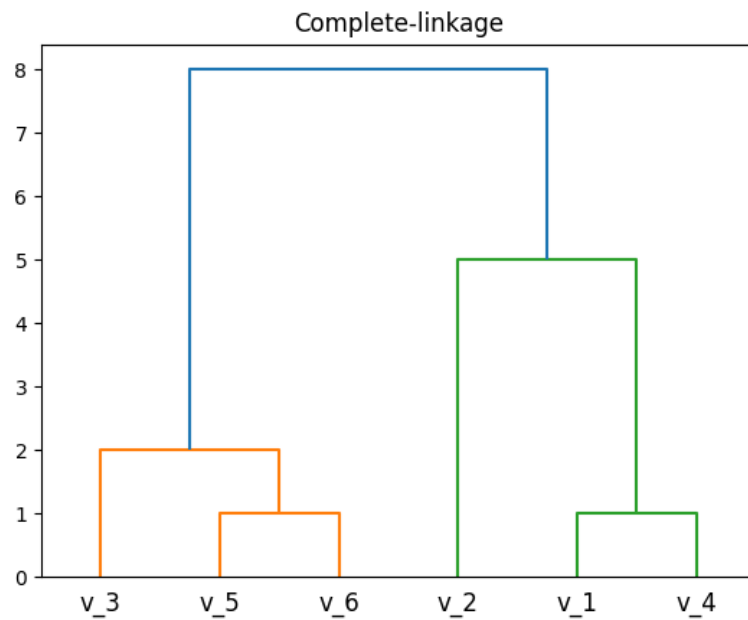


Figure 4.12: The dendrogram corresponding to the graphs in Figure 4.10 using complete-linkage.